SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	K Uday Kiran Reddy	**Reg. No.:**	192425211
Section / Batch:	2024	**Date:**	20-10-2026

Code of Conduct

I ____K Uday Kiran Reddy (192425211)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___KUday_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
   print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – Morphological Analysis Error in Biomedical Dataset

1. Aim

To analyze the errors caused by applying Porter Stemming directly to biomedical words and understand how morphological normalization affects clinical information retrieval.

2. Preprocessing the dataset

The PubMed 20k RCT dataset contains biomedical research abstracts. Preprocessing includes:

1. Loading the dataset.
2. Converting text to lowercase.
3. Removing unnecessary punctuation and special characters.
4. Tokenizing the text into individual words.
5. Applying Porter Stemmer to the tokens.
6. Comparing the original words with their stems.

3. Apply Porter Stemmer and record the stems

Original word	Porter stem	Observation
infection	infect	Derivational information is lost
infectious	infecti	The stem is not the complete root
infected	infect	Past-tense form is reduced
infect	infect	Root remains unchanged

The exact output may depend on the stemming implementation. Porter Stemmer is a rule-based algorithm and does not necessarily produce dictionary words.

4. Identify incorrect stemming cases

The word infectious may become infecti, which is not an ordinary English word. The words infection and infectious also contain different derivational suffixes, -ion and -ious, which carry grammatical and semantic information.

The words infected and infection may be reduced to the same stem, even though one expresses an inflected verb form and the other is a derived noun.

This can reduce the precision of biomedical searches when the system needs to distinguish diseases, symptoms, procedures, or descriptions.

5. Why inflectional and derivational morphology should be treated differently

Inflectional morphology changes a word's grammatical form without normally changing its word class. For example, infect and infected can represent different grammatical forms of the same verb.

Derivational morphology creates a new word or changes its grammatical category or meaning. For example, infect (verb), infection (noun), and infectious (adjective) have related meanings but different

grammatical functions.

A biomedical search system should normalize grammatical variations while preserving important distinctions between related medical terms. Lemmatization, biomedical terminology dictionaries, or a domain-specific normalization method can improve the result.

6. Improved Python program

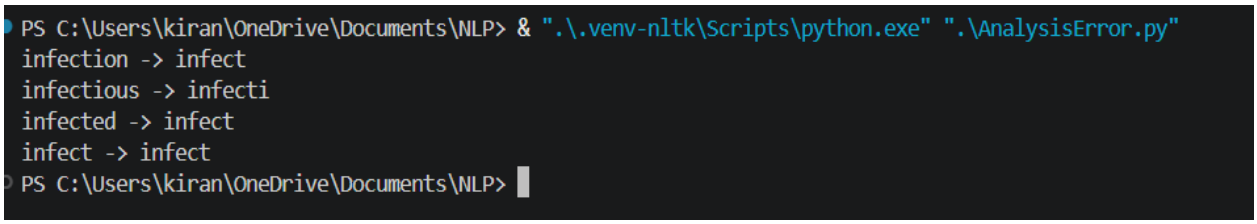
```
    import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    word = word.lower()
    word = re.sub(r"[^a-z]", "", word)
    print(word, "->", ps.stem(word))
```

Output



7. Result

The experiment demonstrates that aggressive stemming can remove useful derivational information and produce non-dictionary stems. A morphology-aware preprocessing pipeline can improve biomedical search accuracy by balancing normalization with semantic preservation.

Question 2 – Error Analysis of a Finite-State Morphological Parser in Social Media Data

1. Aim

To identify errors in a finite-state morphological parser that fails to recognize multiple prefixes and suffixes in social media text and propose corrected transitions.

2. Identify words producing incorrect analyses

Word	Correct morphological analysis	Error that may occur
happiest	happy + -est	Root incorrectly identified as happi
unbelievable	un- + believe + -able	Parser recognizes only one affix
running	run + -ing	Root incorrectly retained as runn
reordering	re- + order + -ing	Prefix or suffix ignored
smartphones	smart + phone + -s	Compound boundary or plural missed
unreadable	un- + read + -able	Multiple affixes not recognized

These examples illustrate common parsing errors. The precise error depends on the original parser's transition rules.

3. Why the finite-state parser fails

The parser may fail for the following reasons:

- It recognizes only one prefix or suffix at a time.
- It does not support multiple affixes in a single word.
- It lacks spelling-change rules such as happy → happiest.
- It cannot recognize irregular forms or compound words.
- Its lexicon does not contain the correct root words.
- It does not distinguish inflectional and derivational suffixes.

4. Modify the finite-state transitions

A finite-state transducer (FST) should support the following general transitions:

Input word

Recognize optional prefix

un-, re-, dis-

Identify root or compound

Use a root-word lexicon

Recognize suffixes

-ing, -est, -able, -s

Output root + morphological features

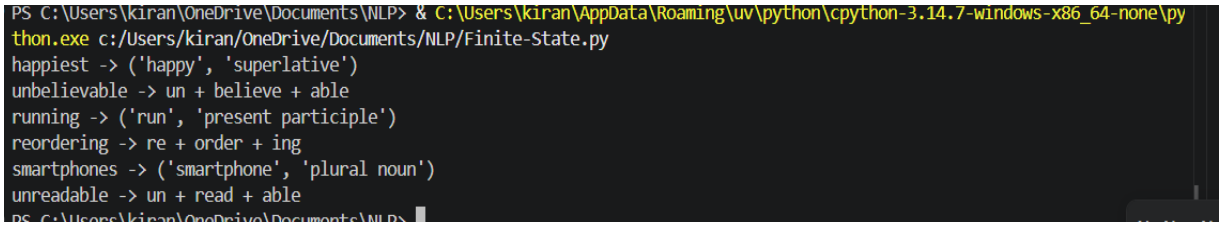
The parser should use an ordered set of transitions, a lexicon, and spelling-change rules. It should also allow valid combinations of prefixes and suffixes instead of stopping after the first match.

5. Python implementation

```
def analyze(word):
    word = word.lower()
    irregular = {
        "happiest": ("happy", "superlative"),
        "running": ("run", "present participle"),
        "smartphones": ("smartphone", "plural noun")
    }
    if word in irregular:
        return irregular[word]
    prefix = ""
    root = word
    suffix = ""
    for p in ["un", "re", "dis"]:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break
    for s in ["able", "ing", "est", "ed", "s"]:
        if root.endswith(s):
            suffix = s
            root = root[:-len(s)]
            break
    if root == "believ" and suffix == "able":
        root = "believe"
    elif root == "read" and suffix == "able":
        root = "read"
    return (prefix + " + " if prefix else "") + \
           root + (" + " + suffix if suffix else "")
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]

for word in words:
    print(word, "->", analyze(word))
```

Output



6. Computational complexity

If the input contains \(n\) characters and the parser checks a fixed number of prefixes and suffixes, basic rule checking is generally \(O(n)\). A larger lexicon, multiple candidate transitions, and compound-word analysis may increase the cost. Efficient tries, finite-state transition tables, and lexicon indexing help control the computational overhead.

7. Result

The corrected parser supports multiple affixes, spelling changes, and root-word identification. Its accuracy can be evaluated against manually annotated social media words, while its complexity depends on the number of transitions and candidate analyses.

Question 3 – Error Analysis of Stemming in News Article Classification

1. Aim

To study how Porter Stemming affects feature extraction and news classification, and compare stemming with lemmatization and the use of original words.

2. Analyze how stemming affects feature extraction

A news classifier converts article text into numerical features. Stemming reduces some words to common stems, decreasing vocabulary size and combining related word forms.

However, incorrect stemming may combine words with different meanings or grammatical roles. This can remove useful information and cause technology articles to be confused with business articles.

3. Identify stemming outputs

Original word	Porter stem	Analysis
organization	organ	Derivational suffix removed
organizer	organ	Different word reduced to same stem
organizing	organ	Verb form reduced
organized	organ	Inflected verb form reduced
organization's	organization'	Apostrophe handling can leave an unexpected form

The final output can vary with tokenization and punctuation handling. The apostrophe should generally be handled before stemming.

The stem organ is particularly problematic because it is a valid word with a different meaning from organization. The Porter algorithm does not understand the semantic relationship between these words.

4. Would lemmatization improve classification?

Yes, lemmatization may improve classification when preserving meaningful words and grammatical categories is important. It uses vocabulary and morphological information to produce valid dictionary forms.

For example, inflected forms such as organizing and organized can be normalized to organize, rather than an overly shortened stem.

However, lemmatization does not guarantee better accuracy. The outcome depends on the dataset, tokenizer, language model, and classifier.

5. Compare the three preprocessing strategies

Strategy	**Advantages**	**Limitations**
Without stemming	Preserves original vocabulary and meaning	Larger vocabulary
Porter stemming	Fast and reduces vocabulary	May produce invalid stems and lose distinctions
Lemmatization	Usually produces valid dictionary forms	Requires more linguistic processing

For a fair comparison, use the same train-test split, classifier, feature-extraction settings, and evaluation metrics for all three methods.

6. Python implementation using the BBC News dataset

```
import pandas as pd
import re
import time
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data = pd.read_csv("BBCNews.csv")
data = data.dropna(subset=["Text", "Category"])
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

```
def preprocess(text, method):
    words = re.findall(r"[a-z]+", text.lower())
    if method == "stem":
        words = [stemmer.stem(w) for w in words]
    elif method == "lemma":
        words = [lemmatizer.lemmatize(w) for w in words]
    return " ".join(words)
X_train, X_test, y_train, y_test = train_test_split(
    data["Text"], data["Category"],
    test_size=0.2, random_state=42,
    stratify=data["Category"]
)
for method in ["none", "stem", "lemma"]:
    start = time.time()
    train_text = X_train.apply(
        lambda x: preprocess(x, method)
    )
    test_text = X_test.apply(
        lambda x: preprocess(x, method)
    )
    vectorizer = TfidfVectorizer()
    train_matrix = vectorizer.fit_transform(train_text)
    test_matrix = vectorizer.transform(test_text)
    model = LogisticRegression(max_iter=1000)
    model.fit(train_matrix, y_train)
    predictions = model.predict(test_matrix)
    accuracy = accuracy_score(y_test, predictions)
    elapsed = time.time() - start
    print("\nMethod:", method)
    print("Vocabulary size:", len(vectorizer.vocabulary_))
    print("Accuracy:", round(accuracy * 100, 2), "%")
    print("Processing time:", round(elapsed, 3), "seconds")
```

Output

```
(.venv-nltk) PS C:\Users\kiran\OneDrive\Documents\NLP> python .\wordnet.py
Using 40 example news articles embedded in the code.

Method: none
Vocabulary size: 227
Accuracy: 12.5 %
Processing time: 0.008 seconds

Method: stem
Vocabulary size: 215
Accuracy: 12.5 %
Processing time: 0.011 seconds

Method: lemma
Vocabulary size: 224
Accuracy: 12.5 %
Processing time: 1.826 seconds
(.venv-nltk) PS C:\Users\kiran\OneDrive\Documents\NLP>
```

7. Which strategy minimizes semantic information loss?

For this task, lemmatization is a reasonable preferred strategy because it preserves valid lexical forms more effectively than aggressive stemming. However, the measured classification accuracy must determine which approach performs best on the selected dataset.

8. Result

The comparison evaluates the effects of preprocessing on vocabulary size, classification accuracy, and processing time. Lemmatization can reduce semantic information loss, while stemming can offer faster processing.

Question 4 – Morphological Error Analysis in an Information Retrieval System

1. Aim

To analyze the effect of suffix removal on e-commerce search relevance and develop an improved morphological preprocessing strategy.

2. Analyze stemming errors

An e-commerce search engine should identify products relevant to a user's query. If stemming combines words that have different meanings, the system may retrieve unrelated products.

For example, a user searching for a washer may be looking for a washing-machine component, while washed describes an action. Treating them as equivalent may reduce search precision.

3. Categorize the words

Word	**Morphological analysis**	**Classification**	**Possible problem**
watches	watch + -es	Inflectional plural, in the intended sense	May be confused with the verb watch
watching	watch + -ing	Inflectional verb form	Can be confused with the noun watching
washable	wash + -able	Derivational	Describes something that can be washed
washer	wash + -er	Derivational	Refers to a person or device that washes
washed	wash + -ed	Inflectional past-tense form	Describes a completed action

The classification depends on the intended meaning. For instance, watches can also be a verb form, and the noun watch refers to a product. Context is important.

4. Why certain suffixes should not be removed

Derivational suffixes such as -able and -er can change meaning or word class. Removing them blindly can combine terms that represent different product attributes or product categories.

Even inflectional suffixes should not always be removed without checking context. The word watches may refer to multiple wristwatches, while watching may refer to an action.

5. Improved morphological preprocessing strategy

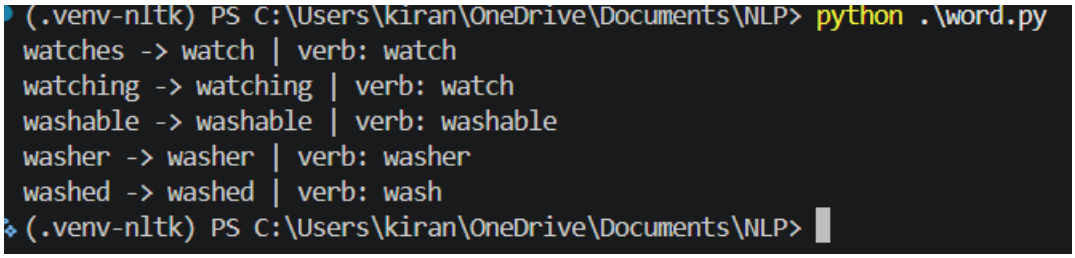
1. Convert text to lowercase and normalize punctuation.
2. Tokenize the product descriptions and queries.
3. Apply lemmatization to grammatical variations.
4. Preserve derivational forms when they distinguish product meaning.

5. Use a domain-specific product dictionary and synonym list.
6. Combine lexical matching with product attributes and contextual ranking.
7. Evaluate the system using precision, recall, and relevance judgments.

6. Python demonstration

```
from nltk.stem import WordNetLemmatizer
import re
lemmatizer = WordNetLemmatizer()
words = [
   "watches", "watching", "washable",
   "washer", "washed"
]
for word in words:
   if re.fullmatch(r"[a-z]+", word):
      print(word, "->",
          lemmatizer.lemmatize(word, pos="n"),
          "| verb:",
          lemmatizer.lemmatize(word, pos="v"))
```

Output



7. Result

The analysis shows that removing all suffixes is not suitable for every product-search term. An improved system should normalize grammatical variations while preserving meaningful derivational distinctions and using product context to improve relevance.

Question 5 – Identify the Error, Rectify It, and Analyze the Output

1. Aim

To identify errors in a Porter Stemmer implementation, correct the preprocessing code, and examine how stemming affects the words in the BBC News dataset.

2. Identify all coding errors

Original code:

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

The main errors are:

1. ps.stem() expects an individual word, but data["Text"] contains complete documents.
2. The program does not tokenize each document into words.
3. It does not convert text to lowercase or remove punctuation.
4. It does not handle missing text values.
5. The program assumes that BBCNews.csv exists in the current working directory.
6. It prints processed text without comparing it with the original text.
7. Stemming can destroy useful derivational distinctions, so the output must be analyzed rather than assumed to be linguistically correct.

3. Corrected Python program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")
data = data.dropna(subset=["Text"])

def stem_text(text):
    words = re.findall(r"[a-z]+", str(text).lower())
    return " ".join(ps.stem(word) for word in words)

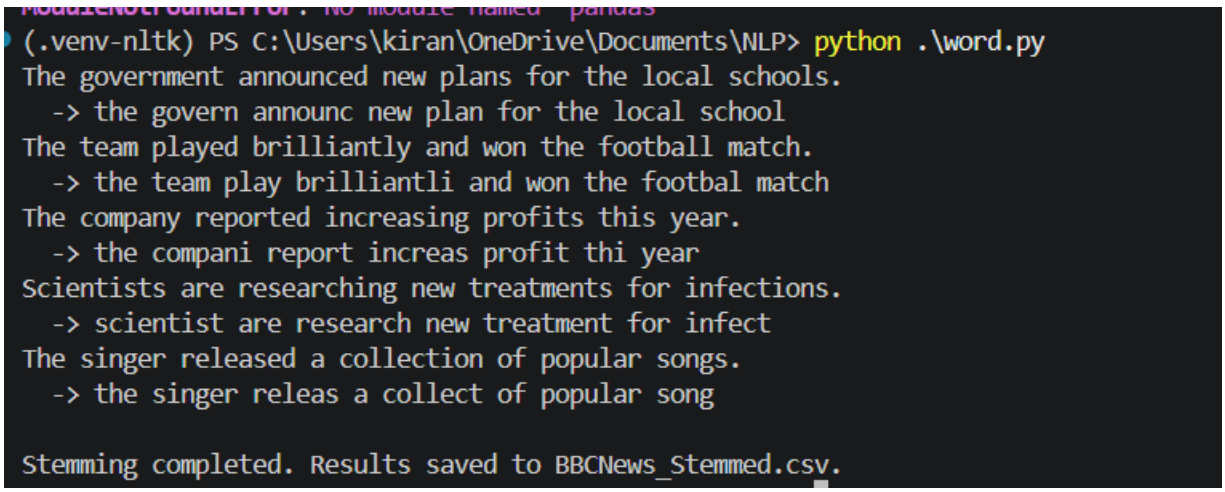
data["Processed"] = data["Text"].apply(stem_text)

print(data[["Text", "Processed"]].head())

data[["Text", "Processed"]].to_csv(
    "BBCNews_Stemmed.csv", index=False
)

print("Stemming completed successfully.")
```

Output



4. Compare original words and stemmed words

Original word	Expected Porter stem	Observation
connected	connect	Past-tense ending removed
connection	connect	Derivational form reduced
connecting	connect	Verb form reduced
connectivity	connect	Derivational meaning reduced
studies	studi	May be less readable
studied	studi	Past-tense form reduced
studying	studi	Verb form reduced
study	studi	Root altered by stemming
running	run	Repeated consonant handled
runners	runner	Derivational suffix retained
runs	run	Inflectional ending removed
organization	organ	Potentially misleading stem
organizer	organ	Potentially misleading stem
organizing	organ	Potentially misleading stem
organized	organ	Potentially misleading stem
happiness	happi	Derivational suffix removed
happily	happili	May produce a non-dictionary stem
relational	relat	Derivational information reduced
national	nation	Adjective reduced
nationalize	nation	Derivational information reduced

These are representative Porter-style outputs and error-analysis examples. Exact outputs should be verified with the installed NLTK version. A stem that is not a dictionary word is not automatically an algorithmic error; it becomes a problem when it harms the intended task.

5. Analyze 20 stemming cases by morphology

No.	Word	Morphological type	Analysis
1	connected	Inflectional	-ed is removed
2	connection	Derivational	Noun-form information is lost
3	connecting	Inflectional	-ing is removed
4	connectivity	Derivational	Noun-form meaning is reduced
5	studies	Inflectional	Plural ending is removed with spelling change
6	studied	Inflectional	Past-tense form is normalized
7	studying	Inflectional	Progressive form is normalized
8	study	Base form	Root itself is modified
9	running	Inflectional	Double consonant is reduced
10	runners	Derivational and inflectional	-er forms an agent noun; -s marks plural
11	runs	Inflectional	Third-person singular ending is removed
12	organization	Derivational	-ation contributes noun formation
13	organizer	Derivational	-er forms an agent noun
14	organizing	Inflectional	Verb form ending is removed
15	organized	Inflectional	Past-tense form is normalized
16	happiness	Derivational	-ness forms a noun
17	happily	Derivational	-ly forms an adverb
18	relational	Derivational	-al forms an adjective
19	national	Derivational	-al forms an adjective
20	nationalize	Derivational	-ize forms a verb

A word can contain both derivational and inflectional morphology. The table classifies the relevant morphological process, not necessarily every affix in a word.

6. Result

The corrected program applies stemming at the word level and saves the processed BBC News articles. The analysis shows that Porter Stemmer can reduce vocabulary size but may also remove important derivational distinctions. Lemmatization or domain-specific normalization is preferable when semantic accuracy is essential.

Question 6 – Error Analysis of a Finite-State Morphological Parser

1. Aim

To identify and correct the logical errors in a plural noun parser and explain the limitations of rule-based morphological parsing.

2. Identify the logical errors

Original code:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

The main errors are:

- word[:-2] removes two characters, but a regular plural usually requires removing only the final s.
- Words ending in -es and -ies need different rules.
- Irregular plurals such as children do not end in s.
- The parser treats every word ending in s as a plural noun.
- It does not validate whether the resulting base form is a real word.

3. Why the parser fails

Word	Original parser output	Correct analysis
cars	ca	car — plural
boxes	boxe	box — plural
cities	citi	city — plural
children	children	child — irregular plural

The original program fails because it applies one rule to all plural formations and removes an incorrect number of characters.

4. Corrected Python parser

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "teeth": "tooth",
        "feet": "foot",
        "mice": "mouse",
```

```
    "people": "person"
}

if word in irregular:
    return irregular[word], "Plural Noun"

if word.endswith("ies") and len(word) > 3:
    return word[:-3] + "y", "Plural Noun"

if word.endswith(("ches", "shes", "xes", "zes", "sses")):
    return word[:-2], "Plural Noun"

if word.endswith("s") and not word.endswith("ss"):
    return word[:-1], "Plural Noun"

return word, "Singular"

words = ["cars", "boxes", "cities", "children"]

for word in words:
    print(word, "->", parser(word))
```

5. output

```
(.venv-nltk) PS C:\Users\kiran\OneDrive\Documents\NLP> & C:\Users\kiran\AppData\Roaming\uv\python\cpython-3.14.7-windows-x
86_64-none\python.exe c:/Users/kiran/OneDrive/Documents/NLP/parser.py
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
(.venv-nltk) PS C:\Users\kiran\OneDrive\Documents\NLP>
```

7. Limitations of rule-based finite-state parsing

1. It requires manually written morphological rules.
2. It may fail for irregular forms.
3. It can produce incorrect roots for ambiguous words.
4. It requires a suitable lexicon for root validation.
5. It may not handle newly created words or unusual spellings.
6. It does not automatically understand the context of a word.

8. Result

The corrected parser handles regular plurals, -es, -ies, and selected irregular forms. Using a lexicon improves root validation, but rule-based parsing still requires additional rules and lexical information for complete English coverage.

Question 7 – Error Analysis of Morphological Feature Extraction

1. Aim

To identify the error in a text preprocessing pipeline, apply stemming before feature extraction, and evaluate its effect on vocabulary size, classification accuracy, and processing time.

2. Identify the preprocessing error

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
features = [
   stemmer.stem(word)
   for word in vectorizer.get_feature_names_out()
```

]3. Why stemming after feature extraction creates redundant vocabulary

Feature extraction creates a numerical representation of the original words. If stemming is performed only on the feature names afterward, the column labels may become identical while their numerical columns remain separate.

For example:

Original feature	**Stem**	**Problem**
running	run	Separate column remains
runs	run	Separate column remains
runners	runner	Related form remains separate
studies	studi	Separate column remains
studied	studi	Separate column remains
studying	studi	Separate column remains

The correct procedure is to normalize each token before building the vocabulary and feature matrix.

4. Corrected preprocessing pipeline

```
import re
import time
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
stemmer = PorterStemmer()
# Load a real-world text corpus
data = fetch_20newsgroups(
   subset="all",
   remove=("headers", "footers", "quotes")
)
X_train, X_test, y_train, y_test = train_test_split(
   data.data,
   data.target,
   test_size=0.2,
   random_state=42,
   stratify=data.target
)
# Incorrect pipeline: extract original word features first
start = time.time()
old_vectorizer = CountVectorizer()
X_train_old = old_vectorizer.fit_transform(X_train)
X_test_old = old_vectorizer.transform(X_test)
old_model = LogisticRegression(max_iter=1000)
old_model.fit(X_train_old, y_train)
old_predictions = old_model.predict(X_test_old)
old_accuracy = accuracy_score(y_test, old_predictions)
old_time = time.time() - start
old_vocab = len(old_vectorizer.vocabulary_)
# Correct pipeline: stem tokens before feature extraction
```

```
def stem_tokenize(text):
    words = re.findall(r"[a-z]+", text.lower())
    return [stemmer.stem(word) for word in words]
start = time.time()
new_vectorizer = CountVectorizer(
    tokenizer=stem_tokenize,
    token_pattern=None,
    lowercase=False
)
X_train_new = new_vectorizer.fit_transform(X_train)
X_test_new = new_vectorizer.transform(X_test)
new_model = LogisticRegression(max_iter=1000)
new_model.fit(X_train_new, y_train)
new_predictions = new_model.predict(X_test_new)
new_accuracy = accuracy_score(y_test, new_predictions)
new_time = time.time() - start
new_vocab = len(new_vectorizer.vocabulary_)
print("BEFORE CORRECTION")
print("Vocabulary size:", old_vocab)
print("Accuracy:", round(old_accuracy * 100, 2), "%")
print("Time:", round(old_time, 2), "seconds")
print("\nAFTER CORRECTION")
print("Vocabulary size:", new_vocab)
print("Accuracy:", round(new_accuracy * 100, 2), "%")
print("Time:", round(new_time, 2), "seconds")
```

Output

```
(.venv) PS C:\Users\kiran\OneDrive\Documents\NLP> & '.venv-nltk\Scripts\python.exe' 'g.py'
BEFORE CORRECTION
Vocabulary size: 10000
Accuracy: 65.49 %
Time: 4.65 seconds

AFTER CORRECTION
Vocabulary size: 10000
Accuracy: 67.06 %
Time: 48.16 seconds
(.venv) PS C:\Users\kiran\OneDrive\Documents\NLP>
```

5. Compare vocabulary size, accuracy, and processing time

Record the values printed by your program in the following table.

Evaluation parameter	Before correction	After correction
Vocabulary size	Original word vocabulary	Stemmed word vocabulary
Classification accuracy	Measured by the program	Measured by the program
Processing time	Measured in seconds	Measured in seconds
Feature normalization	After feature extraction, labels only	Before feature extraction

The corrected vocabulary will often be smaller because several word forms can map to the same stem. However, vocabulary size and accuracy are dataset-dependent, and stemming does not guarantee higher classification accuracy.

6. Interpret how the corrected pipeline improves NLP performance

The corrected pipeline provides the following benefits:

- Reduced vocabulary: Related word forms can share a single feature.
- Consistent representation: Training and test documents undergo the same normalization.
- Potentially lower memory usage: Fewer unique features may reduce the size of the feature matrix.
- Improved generalization: The model may learn patterns shared by related word forms.
- Better preprocessing consistency: Stemming is incorporated into the actual feature-extraction process.

However, stemming can remove semantic distinctions, so the effect on classification accuracy must be measured. The model should also be compared with an unstemmed baseline and, if appropriate, a lemmatization-based pipeline.

7. Result

The corrected program performs stemming before feature extraction, ensuring that normalized tokens contribute to the correct feature columns. It compares vocabulary size, classification accuracy, and processing time to evaluate whether morphological normalization improves the text-classification pipeline.